Plan: Galerkin Linear-Vortex Panel Solver (Rigid Wake Phase)

Oct 12, 2026 · @Mark

Purpose and scope

This phase proves the core scheme: a Galerkin panel method with nodal doublet strengths and linear vortex-sheet elements, robust to skewed elements and small mesh gaps, on a steady problem with a rigid wake. The vortex particle method is deferred, but every interface is designed so it slots in later.

Locked decisions for this phase

- Steady flow, no body motion. Assemble and solve once.
- Closed thick bodies only. Open tips are auto-capped by the solver (section 3). No zero-thickness surfaces.
- Non-intersecting meshes. Tolerance-based repair of small gaps, T-junctions and slivers is in scope; intersecting components are not.
- Rigid wake: fixed streamwise strips from the trailing edge, first implicit row fixed in the body frame.
- Galerkin testing with nodal hat functions as the default, with a switch for the alternative test weights.
- Quads split into triangles as the default element; isoparametric bilinear quads as a second element type to compare.
- Dense matrices and a direct solver. Fast summation is deferred.
- Built inside the existing codebase.

Note to Claude Code. You will be given an existing codebase that implements a Morino-type formulation (constant-strength source and doublet panels, Dirichlet boundary condition). Use it as the reference solver: run both codes on the same reference case and mesh family and compare the quantities in section 11. Before writing new infrastructure, read the existing code for mesh ingestion, trailing-edge detection, wake construction and post-processing, and reuse what fits. Do not modify the Morino solver's numerics; keep it as an independent baseline.

Formulation

The unknowns are scalar doublet strengths μ at mesh nodes, piecewise linear on triangles, with the no-penetration condition tested against the same hat functions.

Unknowns. $\mu(x) = \Sigma_j \mu_j \varphi_j(x)$, where φ_j is the nodal hat function. The unknown is a scalar doublet, not a vector vorticity. That gives one unknown per node and a divergence-free vortex field automatically.

Vortex sheet. On each flat triangle T, $\gamma_T = n_T \times \nabla\mu$ is constant. Node j contributes $\gamma_j|_T = \mp e_j/(2A_T)$, where e_j is the edge opposite node j. Fix the sign convention once and test it.

Edge filaments. Where μ is continuous, edge filaments cancel between neighbours. Filaments survive only where the layer is cut or ends: at the trailing edge (cancelled by the Kutta condition) and at the wake's tip edges and far end.

Sources. Prescribed, not solved for: $\sigma = -V_\infty \cdot n$. The interior is then at rest and μ equals the outer perturbation potential.

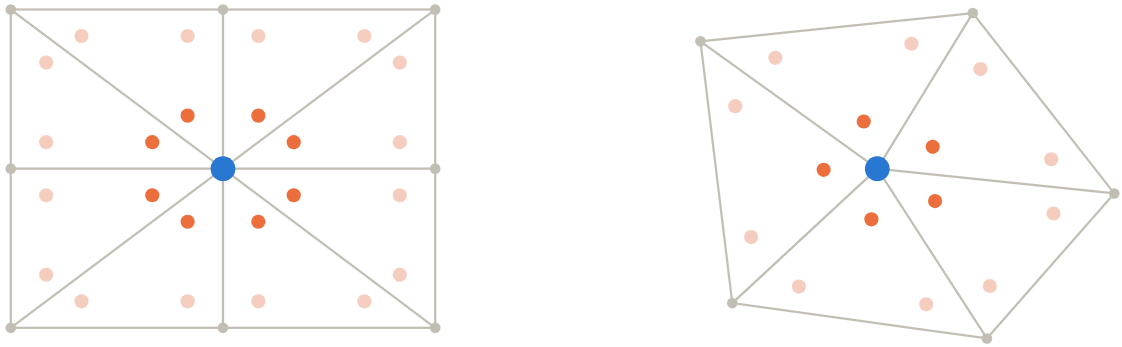
Boundary condition. For every node i:

```
\int_S \varphi_i \,(\mathbf{n}\cdot\mathbf{u})\, dA = 0, \qquad \mathbf{u} =
\mathbf{V}_\infty + \mathbf{u}_\sigma + \mathbf{u}_\gamma[\mu] +
\mathbf{u}_{\text{wake}}[\mu]
```

Trial and test spaces are the same, so the system is square. The particle velocity term is added to u when the VPM is coupled; the assembly takes it as an optional right-hand-side contribution from day one.

Each node unknown μ_i is tested against its hat function over its one-ring

Quads are split into triangles; unstructured triangles are used as given



Split quads **Unstructured triangles**

Node i: unknown scalar doublet μ_i, one per node

Quadrature points weighted $W \cdot \varphi_i$: darker means larger φ_i

Row i: $\int \varphi_i \, (n \cdot u) \, dA = 0$ over the one-ring; each triangle is a constant sheet $\gamma = n \times \nabla\mu$

Nodal unknowns and Galerkin test weights · split quads and unstructured triangles

Each node owns one boundary-condition row, integrated over its one-ring with quadrature points weighted by its own hat function.

Geometry and topology layer

The geometry layer turns an imperfect surface mesh into a closed, consistently oriented triangulation with classified edges. It never rejects a mesh; it repairs, logs and reports quality.

1. **Ingest** quads and triangles, reusing the existing codebase's readers where possible.
2. **Split quads** into two triangles along the shorter diagonal, or the diagonal with the smaller fold angle on warped quads. Keep the parent-quad id so the isoparametric element can be swapped in later.
3. **Weld nodes** within a tolerance relative to the local edge length (start at about 1e-3 of the shortest incident edge; make it configurable).
4. **Build half-edge connectivity** and orient all normals outward. Fail loudly on non-manifold edges, since intersecting meshes are out of scope.
5. **Detect T-junctions**: nodes lying on another element's edge within tolerance. Record them for constraints in section 4.
6. **Classify open boundary loops:**

    - *Small gap*: two nearly coincident boundary chains. Stitch them with virtual edges so μ stays continuous across the gap.
    - *Opening*: a closed loop enclosing an area, such as an uncapped wing tip. Auto-cap it.
7. **Auto-cap** each opening with no new nodes. For a wing tip, pair the upper and lower polylines from the leading edge to the trailing edge and build a ladder of triangles between them. Cap triangles are ordinary panels: they get sources, sheet velocity and boundary-condition test integrals. Keep a debug flag to leave openings uncapped, and an option to zip instead of cap, for comparison.
8. **Detect Kutta edges** with a dihedral-angle test plus a user override list. Reuse the existing codebase's trailing-edge detection if it is suitable.
9. **Report mesh quality**: minimum angle, aspect ratio, skew, sliver count, gap and T-junction counts, number of capped openings.

DOF map and constraint matrix C

Every mesh pathology and the Kutta condition become rows of one sparse matrix C mapping reduced unknowns q to nodal doublet values: μ_nodes = C q. This is the main robustness device and should be the single place where topology decisions live.

- **Welded or stitched nodes** share one entry of q.
- **T-junction hanging nodes** are linear interpolations of the two endpoints of the long edge.
- **Trailing-edge nodes** are duplicated into upper and lower copies, each a separate entry of q. Upper-surface triangles reference the upper copy, lower-surface triangles the lower copy. Cap triangles at the tip's TE node reference one copy (choose one and document it).
- **Wake nodes** are not unknowns. Their μ is a row of C built from the Kutta condition (section 7).

The reduced Galerkin system is $(C^T A C)\, q = C^T b$. Applying C^T to the test side keeps the method a true Galerkin method. C should be built and unit-tested independently of the kernels: check that every row sums correctly and that a linear μ field is reproduced exactly through T-junctions and stitches.

Kernels

One analytic kernel, the velocity of a unit-strength source triangle, supplies every influence in the method.

```
\mathbf{w}_{T'}(\mathbf{x}) = \frac{1}{4\pi}\int_{T'} \frac{\mathbf{x}-
\mathbf{y}}{|\mathbf{x}-\mathbf{y}|^3}\, dA_y
```

- **Sources**: $u_\sigma = \sigma_{T'}\, w_{T'}(x)$.
- **Vortex sheet**: for constant γ on T', $u_\gamma(x) = \gamma \times w_{T'}(x)$.

Use the closed-form flat-triangle formula (Hess–Smith or Newman). Reuse the existing codebase's source-panel routine if it returns the full velocity vector for triangles.

Requirements

- Generic over the number type, so ForwardDiff works later.
- Correct limits on the panel's own plane: the normal component jumps by ±½ across the panel, and the tangential components are log-singular at edges and vertices. Return the limit from a specified side.
- A far-field switch (point source or multipole) beyond a configurable distance, verified for continuity at the switch.
- Unit tests against brute-force adaptive quadrature at far, near, in-plane, edge-adjacent and vertex-adjacent points, including slivers and highly skewed triangles.

Galerkin assembly and quadrature

Assembly loops over pairs of a test triangle T on the body and a source triangle T' on the body or wake, producing a 3×3 local block that is scattered into a global matrix A over nodal μ.

At each quadrature point x_q of T, evaluate $w_{T'}(x_q)$ once, then for local nodes a on T and b on T':

```
A_{ab} \mathrel{+}= W_q\, \varphi_a(\mathbf{x}_q)\; \mathbf{n}_T \cdot
\big(\boldsymbol{\gamma}_b \times \mathbf{w}_{T'}(\mathbf{x}_q)\big)
```

Rows are body nodes only; the wake is never tested. Columns cover body and wake nodes, and C folds the wake columns back onto the body unknowns.

Quadrature tiers, chosen by distance between T and T' relative to panel size:

Tier	Condition	Outer rule on T
Far	more than about 3 panel diameters apart	1–3 point Gauss
Near	closer, not touching	6–7 point Gauss, or one level of subdivision
Touching	self, shared edge or shared vertex	Subdivision graded toward the shared entity (or Duffy transform); the inner integral is analytic, so the outer integrand is only log-singular

Thresholds and orders are configurable. Converge them on the sphere test before anything else.

Right-hand side. $b_i = -\int \varphi_i\, n\cdot(V_\infty + u_\sigma)\, dA$. The freestream term is exact: each triangle adds $V_\infty \cdot n_T A_T/3$ to each of its vertices. The source term uses the same w kernel and quadrature tiers. Leave a hook for an external velocity field (particles) evaluated at the same quadrature points.

Test-weight switch. Make the test weight a parameter: hat function φ_i (default), one-ring indicator, or median dual-cell indicator. Same loop, same points; only the weight changes.

Element interface. Define basis(elem, ξ), sheet_velocity(elem, x) and quadrature(elem, tier), so the isoparametric bilinear quad can be added later without touching assembly. That element needs numerical inner integrals and singular quadrature on touching quad pairs.

Kutta condition and rigid wake

The wake is a fixed set of streamwise strips of linear-μ triangles attached to the trailing-edge nodes, with the nodal Kutta condition $\mu_w = \mu_u - \mu_l$.

Nodal Kutta condition. At each TE node k, $\mu_{w,k} = \mu_{u,k} - \mu_{l,k}$ (sign fixed by the orientation convention). Because μ is linear along each TE edge, matching at nodes cancels the TE filament along the whole edge, not just on average.

Non-matching upper and lower TE nodes. Use the union of both node sets as the wake's TE nodes. At each union node, interpolate μ_u linearly along the upper TE edges and μ_l along the lower. Cancellation stays exact, because a piecewise-linear function on a partition remains piecewise linear on any refinement. The interpolation weights go into C.

Wake geometry.

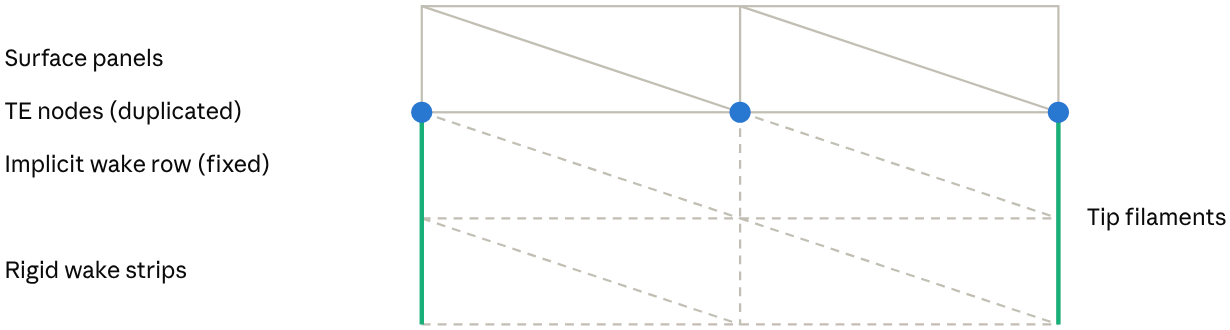
- Strips leave each wake TE node along the freestream (option: along the TE bisector for the first row).
- The first row is fixed in the body frame; its length is a parameter.
- Downstream rows continue along the freestream with spacing growing geometrically.
- Every node in a strip carries the same $\mu_{w,k}$, so the wake carries trailing vorticity only.
- Truncate 20–50 spans downstream, or close each strip with a semi-infinite end element. Either way there must be no spurious starting-vortex filament at the far end. Compare both on the reference case.

Wake tips. The outer side edges of the wake carry real filaments of strength μ_w; include them in the wake velocity. At the outermost TE node, the residual jump on the body runs along the cap edge assigned in section 4. Check tip loading under refinement.

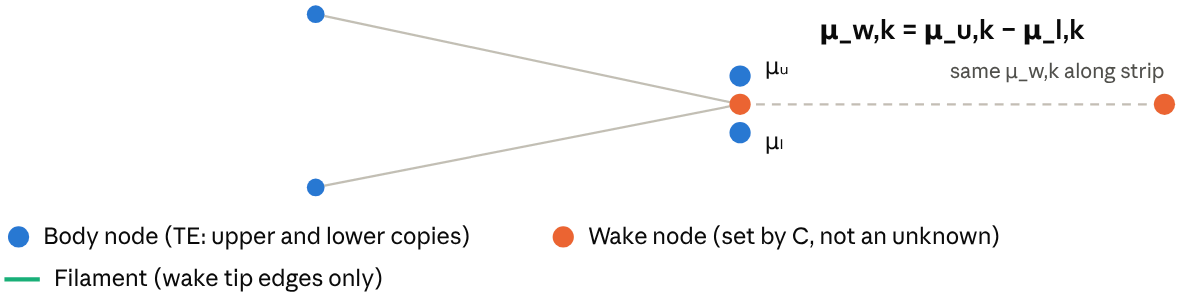
Filament rule. Edge filaments exist only at the true boundary of the combined body-and-wake layer. Interior edges, TE edges satisfying the Kutta condition and (later) the interface edge at particle conversion carry none.

The wake value at each TE node is the jump between its upper and lower copies

Plan view



Section through one TE node



Upper and lower copies are separate unknowns, each tested only over the panels on its own side.

Matching μ at TE nodes cancels the TE filament along whole edges (μ is linear along each edge).

Duplicated TE nodes, nodal Kutta condition and rigid wake strips

The TE copies are unknowns; the wake values follow from them through C, and only the wake tip edges keep filaments.

Linear system

On a closed body, constant μ induces no velocity, so the reduced matrix is singular; solve a bordered system with one constraint row per closed body.

```
\begin{bmatrix} C^\top A C & \mathbf{e} \\ \mathbf{c}^\top & 0 \end{bmatrix}
\begin{bmatrix} \mathbf{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} C^\top
\mathbf{b} \\ 0 \end{bmatrix}
```

- $c = C^T(\int \varphi_i \, dA)$ pins the area-weighted mean of μ to zero; e is the vector of ones on that body's unknowns.

- The multiplier λ absorbs the small incompatibility from quadrature error. Report it every solve as a quadrature-quality diagnostic: it should shrink as quadrature is refined.
- The null space survives the Kutta coupling: a constant μ gives equal upper and lower TE values, so μ_w = 0. Lifting bodies need the bordered row too.
- Dense LU for this phase. Report the condition number (estimate) on every test case, and compare it with the Morino code's.

Post-processing and diagnostics

Surface velocity needs no gradient stencils: ∇μ is constant on each triangle, so the tangential perturbation velocity comes straight from the element.

- **Surface velocity** on the outer side of triangle T: V∞ + υ_σ + υ_γ (principal value) + ½ γ_T × n_T, with the jump term's sign verified on the sphere. Equivalently, the tangential velocity is V∞,t + ∇μ_T when μ is the perturbation potential. Compute both and check that they agree.
- **Pressure**: Cp = 1 − |υ|²/V∞², per triangle at its quadrature points. Also provide nodal Cp by area-weighted averaging, for plots.
- **Forces and moments, two independent routes:**
  - pressure integration of Cp over the body;
  - Kutta–Joukowski on the wake and Trefftz-plane induced drag from the wake strips' μ_w. Report both lift values and their difference.
- **Leak diagnostic**: the residual n·υ at quadrature points, reported as RMS and maximum per tier, and the net flux through each capped opening and stitched gap.
- **Mesh-sensitivity outputs**: spanwise loading, sectional Cl, and μ along the trailing edge, written out for direct comparison with the Morino code.

Code architecture

Seven modules, each testable on its own, with topology confined to the DOF map and physics confined to the kernels.

Module	Responsibility	Depends on
Geometry	Ingestion, quad split, welding, half-edges, T-junctions, gap stitching, auto-capping, Kutta-edge detection, quality report	—
DOFMap	TE duplication, constraint matrix C, wake rows from the Kutta condition	Geometry
Kernels	Unit source-triangle velocity w_T, sheet velocity γ × w, far-field switch	—
Elements	Element interface: basis, sheet velocity, quadrature tiers; flat triangle now, isoparametric quad later	Kernels
Wake	Rigid strip construction, tip edges, end treatment	Geometry, DOFMap
Assembly	Galerkin local blocks, test-weight switch, RHS with an external-velocity hook, bordered system, dense solve	Elements, DOFMap, Wake
Post	Surface velocity, Cp, both force routes, leak and quadrature diagnostics, comparison output	Assembly

Where the existing codebase already provides a module's job (mesh reading, TE detection, plotting), wrap it rather than rewriting. Keep types generic over the float type throughout.

Validation plan and Morino comparison

Each stage must pass before the next starts, and every test after stage 1 runs on both a clean mesh and an abused version of it.

1. **Mesh-abuse generator.** From a clean mesh, produce variants with random node skew, T-junctions, small gaps, slivers and opposite-direction skew between upper and lower TE rows. Seeded and reproducible.
2. **Kernel and C unit tests.** w_T against brute-force quadrature; C reproduces linear μ through T-junctions and stitches.

3. **Sphere in uniform flow.** The exact surface μ is linear in the streamwise coordinate. Check convergence of μ and Cp under refinement, the λ diagnostic, the leak residual, and sensitivity to quadrature tiers. Repeat on a cube-sphere quad mesh, split into triangles.
4. **Ellipsoid**, for a non-uniform analytic Cp.
5. **Rectangular wing with rigid wake.** Capped and uncapped versions (auto-cap vs zip vs open). Lift, spanwise loading and induced drag, both force routes, convergence under refinement.
6. **Reference-case comparison with the Morino codebase**, below.
7. **Element comparison**: split triangles vs isoparametric quads on the cube-sphere and the wing, once the quad element exists.
8. **Test-weight comparison**: hat vs one-ring vs dual cell, on the wing with abused meshes.

Reference-case comparison. Run the existing Morino solver and the new solver on the same reference case and the same mesh family (the case provided with the codebase; a NACA 0012 rectangular wing at moderate incidence if none is provided). Compare:

- CL, CDi and pitching moment against refinement, on clean meshes. Both codes should converge to nearly the same answer.
- The same quantities against TE skew magnitude δ, including opposite upper/lower skew, repeating the earlier sensitivity study. The new solver should be much less sensitive; quantify the ratio.
- Symmetric section at zero incidence with skewed TE rows: lift must stay near zero.
- Chordwise Cp at several span stations, and the spanwise loading.
- Robustness to gaps and slivers: whether each code runs, and the change in CL relative to the clean mesh.
- Condition number and wall-clock time at matched resolution.

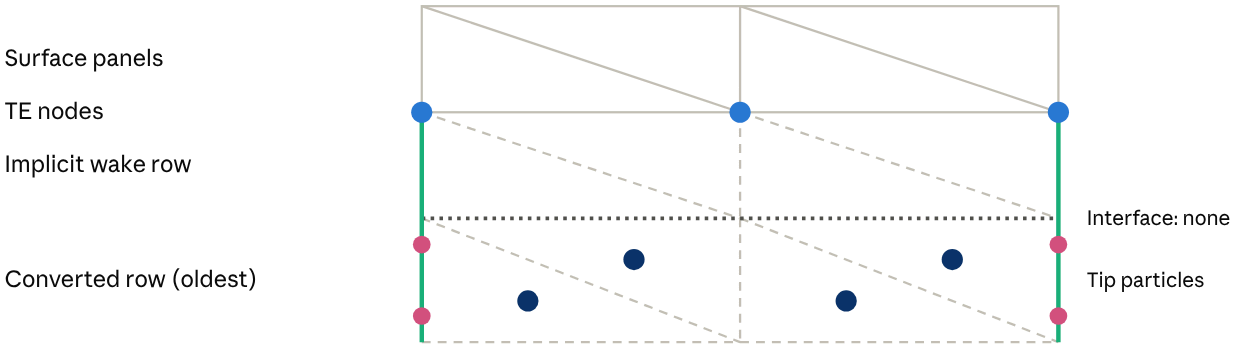
Write results as tables and plots in a reproducible script so the comparison can be rerun after every change.

Deferred items and hooks

These are out of scope for this phase, but the listed hook must exist so adding them later needs no restructuring.

Deferred item	Hook required now
Vortex particle wake: two particles per wake quad (one per triangle, $\alpha_t = \Sigma$ edges $\frac{1}{2}(\mu_a + \mu_b)(x_b - x_a)$ at the centroid), one-particle option, tip-edge particles at 2-point Gauss positions, starting vortex on first conversion, no particles for interface edges	External-velocity hook in the RHS at quadrature points; wake rows stored as convertible units
Unsteady flow and a free near wake	Wake geometry separated from the Kutta rows in C; implicit vs explicit wake rows flagged
Body motion, relative motion	Sources written as $\sigma = -(V_\infty - V_{body})\cdot n$ with $V_{body} = 0$ for now
Isoparametric bilinear quads	Element interface in Elements
Symmetric Galerkin (Nédélec) form	Assembly local-block function isolated so a second form can be added
Fast summation (FMM or H-matrix) with GMRES	Kernel evaluation behind a single interface
Pressure-type Kutta condition	Kutta rows built in one function in DOFMap

Each converted wake triangle becomes one particle; tip edges add two more



- Triangle particle at centroid: $\alpha_t = \Sigma$ over its 3 edges of $\frac{1}{2}(\mu a + \mu b)(xb - xa)$
- Tip-edge particles at 2-point Gauss positions, each carrying half of $\frac{1}{2}(\mu a + \mu b)(xb - xa)$
- Interface edge: no particles and no panel filament on either side

The starting vortex edge is converted on the first conversion only.
One-particle option: the quad edge sum placed at the quad centroid.

Planned particle conversion of the oldest wake row · two particles per quad

The planned VPM conversion is local to each wake triangle; nothing is converted across the interface edge.

Prior art to check

These references were recalled from memory without a literature search, so verify each one before relying on it.

- **Galerkin BEM:** Steinbach, *Numerical Approximation Methods for Elliptic Boundary Value Problems* (Springer, 2008); Sauter & Schwab, *Boundary Element Methods* (Springer, 2011) for touching-pair quadrature.
- **Hypersingular operator and the curl identity:** Nédélec (1982), *Integral Equations and Operator Theory*; Maue (1949).
- **Singularity extraction and singular quadrature:** Wilton et al. (1984) and Graglia (1993) from computational electromagnetics; Duffy (1982), *SIAM Journal on Numerical Analysis*.
- **Loop basis functions** in EM method-of-moments codes: n × ∇(hat function), identical to the γ_j used here; the low-frequency loop-star literature covers their conditioning.
- **Source-panel kernel:** Hess & Smith (1967); Newman (1986), *Journal of Engineering Mathematics*.

- **Reference implementation:** Bempp-cl (Betcke & Scroggs, *JOSS*, 2021) assembles the Laplace hypersingular operator with P1 elements; use it to cross-check matrix entries on a closed non-lifting body.
- **Panel codes:** FLOWPanel.jl, DUST, VSPAERO and PAN AIR, for TE detection, wake handling and Kutta-edge flagging; Willis, Peraire & White (MIT, mid-2000s) for unstructured high-order panels with particle wakes.